



AWSエンジニアのための Cloud Run ハンズオン

Google Cloud / Cloud Run を、AWS との違いから学ぶ

TypeScript + Hono / Artifact Registry / Cloud Run

目次

はじめに

0. 事前準備

座学パート

1. AWSとGoogle Cloudの考え方の違い

2. Dockerのおさらいとコンテナ起動

3. Cloud Runとは

ハンズオンパート

4. Cloud Runへデプロイする

5. 更新とロールバック

6. カナリアリリースとタグ付きURL

7. オートスケールを観察する

8. ログとメトリクスをのぞく

9. 締め: Dockerfileすら書かないデプロイ

発展編(3時間版)

10. 発展編

10-1. Pub/Subとつなぐ

10-2. WebSocketチャット

10-3. Cloud Run Jobs

おわりに

99. 後片付け

AWSエンジニアのための Cloud Run ハンズオン

AWS を実務で使っているエンジニアに向けた、Google Cloud / Cloud Run のハンズオン教材です。

この教材のゴール

1. **Cloud Run** を通して **Google Cloud** の良さを知る — 「デプロイしたら HTTPS の URL が返ってくる」体験から、Google Cloud の SaaS 的なアプローチを体感する
2. **AWS** との違いを知る — 単なるサービス名の読み替えではなく、思想の違いを理解する
3. 別のクラウドの目線を持ち帰る — 普段 AWS で組んでいる構成を「Google Cloud ならどう組むか」で考えられるようになる

対象者

- クラウドを実務で利用したことがある(AWS がメイン)
- `docker build` / `docker run` を打ったことがある、もしくは概念は知っている
- Google Cloud はほぼ触ったことがなくて OK
- サンプルアプリは TypeScript + [Hono](#) 製ですが、書き換えるのは定数2行だけなので TypeScript の経験は不要です

進め方

前半(1〜3章)は座学パートの内容をそのまま収録しています。イベントではスライドで進行しますが、この **GitBook 単体でも最初から最後まで完結する**ように書いてあります。後半(4章〜)は Google Cloud の Cloud Shell エディタだけで完結するハンズオンです。手元へのインストールは不要です。

タイムテーブル

2時間版

時間	内容	章
0:00 - 0:10	オープニング・事前準備の確認	0
0:10 - 0:25	座学: AWS と Google Cloud の考え方の違い	1
0:25 - 0:40	座学: Docker のおさらい ・ Cloud Run とは	2, 3
0:40 - 0:55	ハンズオン: コンテナを作って動かす	2
0:55 - 1:15	ハンズオン: Artifact Registry と Cloud Run へのデプロイ	4
1:15 - 1:30	ハンズオン: 更新とロールバック	5
1:30 - 1:45	ハンズオン: カナリアリリースとタグ付きURL	6
1:45 - 1:55	ハンズオン: オートスケールを観察する	7

時間	内容	章
1:55 - 2:00	ログをのぞく ・ 締め: ソースデプロイ ・ 後片付け	8, 9

3時間版(2時間版に追加)

時間	内容	章
+20分	ハンズオン: Pub/Sub とつなぐ	10
+10分	デモ: WebSocket チャット	10
+10分	デモ or ハンズオン: Cloud Run jobs	10
+20分	各章の深掘り・質疑・バツファ	-

かかる費用

Cloud Run・Artifact Registry とともに無料枠が大きく、このハンズオンの範囲であれば**ほぼ無料枠に収まります**(数円〜数十円程度)。心配な場合は最後の[後片付け](#)を必ず実施するか、ハンズオン専用プロジェクトを作って終了後にプロジェクトごと削除してください。

この教材を手元で表示する

[HonKit](#)(GitBook 互換)でビルドできます。

```
npm install
npm run serve # http://localhost:4000
```

当日用サポートアプリ

[support/](#) に、この GitBook を iframe 表示しつつ「チャット・進捗共有・ヘルプ要請」ができるリアルタイムのサポートアプリがあります(TypeScript + Hono + WebSocket 製で、それ自体を Cloud Run で動かします)。イベントで使う場合は [support/README.md](#) を参照してください。GitBook 単体での利用には不要です。

関連リンク

- [Docker 入門ハンズオン\(introduction-docker\)](#) — Docker をより深く学びたい人向け
- [クラウドを今から学ぶには](#) — 非機能要件・オートスケーリング周りの背景

0. 事前準備

ハンズオン当日までに以下を済ませておいてください。**所要時間は10分程度**ですが、当日に始めると全体が遅れるので必ず事前をお願いします。

用意するもの

- Google アカウント
- クレジットカード(課金の有効化に必要。ハンズオンの範囲はほぼ無料枠に収まります)
- Chrome などのモダンブラウザ

手元のPCへのインストールは一切不要です。ハンズオンはすべてブラウザ上の Cloud Shell エディタで完結します。

1. Google Cloud プロジェクトを作る

AWS と大きく違うポイントその1です。AWS では「アカウント」が課金・リソースの分離単位でしたが、Google Cloud では1つのアカウント(組織)の下に**プロジェクト**をいくつも作り、それが分離単位になります。AWS のマルチアカウント運用 (Organizations) でやっていたことを、Google Cloud はプロジェクトで軽量にやるイメージです。

1. [Google Cloud コンソール](#)にログイン
2. 初めの場合は無料トライアル(90日 / \$300 クレジット)の案内に従う
3. 画面上部のプロジェクトセレクトから「新しいプロジェクト」を作成
 - プロジェクト名: `cloudrun-handson` など(IDは自動でユニークになります)
4. 課金が無効になっていることを確認([課金ページ](#))

ハンズオン専用プロジェクトを推奨します。終わったらプロジェクトごと削除すれば、消し忘れによる課金の心配がありません。

2. Cloud Shell エディタを開く

shell.cloud.google.com を開くか、コンソール右上のターミナルアイコンから Cloud Shell を起動し、「エディタを開く」をクリックします。

Cloud Shell は Google Cloud が無料で提供する開発環境で、AWS の CloudShell に相当しますが、以下が最初から揃っています。

- VS Code ベースのエディタ(AWS CloudShell にはない)
- **Docker** デーモン(AWS CloudShell では使えない。今回のハンズオンの鍵)
- `gcloud` CLI(認証済み)
- 5GB の永続ホームディレクトリ

3. プロジェクトの設定とAPIの有効化

Cloud Shell のターミナルで以下を実行します。`YOUR_PROJECT_ID` は自分のプロジェクトIDに置き換えてください。

```
gcloud config set project YOUR_PROJECT_ID
```

続けて、ハンズオンで使うAPIを有効化します(数分かかることがあるので事前にやっておくのがおすすめです)。

```
gcloud services enable \  
  run.googleapis.com \  
  artifactregistry.googleapis.com \  
  cloudbuild.googleapis.com \  
  pubsub.googleapis.com
```

AWSとの違い: Google Cloud では各サービスの API を明示的に有効化してから使います。AWS のようにすべてのサービスが最初から呼べる状態ではなく、プロジェクトごとに使うサービスをオプトインする思想です。

4. 動作確認

```
gcloud config get-value project  
docker version
```

プロジェクトIDが表示され、`docker version` がエラーにならないければ準備完了です。

トラブルシューティング

- `gcloud services enable` で課金エラーが出る — プロジェクトに課金アカウントが紐付いていません。[課金ページ](#)から紐付けてください。
- **Cloud Shell** がタイムアウトした — 無操作で切断されることがあります。再接続すればホームディレクトリの内容は残っています(環境変数は消えるので再設定してください)。

1. AWSとGoogle Cloudの考え方の違い

ビルディングブロック vs SaaS的アプローチ

AWS の思想は**ビルディングブロック**です。VPC・サブネット・セキュリティグループ・IAMロール・ALB・ターゲットグループ……
小さな部品を積み上げて、自分のアーキテクチャを組み立てます。自由度が高い一方で、「Webアプリを1つ公開する」だけでも組み立てる部品の数は多くなります。

Google Cloud のマネージドサービスは、どちらかというと **SaaS 的なアプローチ**です。「Webアプリを公開したい」という目的に対して、統合済みのサービスが1つ用意されていて、デフォルトで実用的な設定になっています。部品を組む代わりに、完成品の設定を必要な分だけ調整します。

この違いが一番わかりやすいのが、今回扱う Cloud Run です。

例: コンテナを1つ、HTTPSで公開するまで

	AWS (ECS/Fargate)	Google Cloud (Cloud Run)
ネットワーク	VPC・サブネット・SG を用意	不要(必要なら後から VPC 接続)
負荷分散	ALB + ターゲットグループ + リスナー	不要(組み込み)
TLS証明書	ACM で発行し ALB に紐付け	不要(*.run.app のHTTPS URLが自動発行)
オートスケール	Application Auto Scaling を設定	デフォルトで有効(0～自動)
ログ	awslogs ドライバや FireLens を設定	不要(stdout が自動で Cloud Logging へ)
デプロイ	タスク定義 + サービス更新	<code>gcloud run deploy</code> 1コマンド

どちらが優れているという話ではありません。細かく制御したいなら AWS 的なアプローチが強く、素早く価値を出したいなら Google Cloud 的なアプローチが強い。**両方の目線を持っていると、状況に応じて最適な選択ができる**——それが今日のゴールです。

アカウント構造の違い

概念	AWS	Google Cloud
分離の単位	アカウント(Organizationsで束ねる)	プロジェクト(組織の下に複数作る)
リージョンの扱い	コンソールでリージョンを切り替える	リソース作成時に指定(コンソールは全リージョン横断)
権限の主体	IAMユーザー / ロール	Google アカウント / サービスアカウント
API	常に有効	プロジェクトごとに明示的に有効化

特に「プロジェクト」は Google Cloud を触るうえで最初に慣れるべき概念です。dev / staging / prod をプロジェクトで分ける、実験用に使い捨てプロジェクトを作る、といった運用が気軽にできます。

サービス対応表

ハンズオン中に「AWSでいうと何?」と思ったらここに戻ってきてください。

今日使うサービス

Google Cloud	AWSでいうと	補足
Cloud Run	App Runner + Fargate + Lambda	どれとも微妙に違う。次章で詳しく
Artifact Registry (GAR)	ECR	コンテナ以外(npm, Maven等)も置ける
Cloud Shell	CloudShell	エディタ付き・Docker が動く
Cloud Build	CodeBuild	ソースデプロイの裏側で動く
Cloud Logging	CloudWatch Logs	設定不要で stdout を収集
Cloud Monitoring	CloudWatch	メトリクスはデフォルトで収集済み
Pub/Sub	SNS + SQS	1サービスで pub/sub もキューも担う

今日は使わないが対応を知っておくと便利なサービス

Google Cloud	AWSでいうと	補足
Compute Engine	EC2	
GKE	EKS	Kubernetes 発祥の地だけあり完成度が高い
Cloud Functions (Cloud Run functions)	Lambda	現在は Cloud Run 基盤に統合された
Cloud Storage	S3	
Cloud SQL	RDS	
Spanner	(相当なし / 強いて言えば Aurora)	グローバル分散RDB
BigQuery	Redshift + Athena	Google Cloud 最大の看板サービス
Eventarc	EventBridge	各サービスのイベントを Cloud Run へ配送
Cloud Tasks	SQS(遅延キュー用途)	HTTPターゲットに直接push
Cloud Scheduler	EventBridge Scheduler	cron
Secret Manager	Secrets Manager	
Cloud Load Balancing	ALB/NLB + CloudFront の一部	グローバル単一エニーキャストIP
IAM サービスアカウント	IAM ロール	「アカウント」だが実体はロールに近い

まとめ

- AWS は部品を組み立てる。Google Cloud は完成品を調整する
- 分離単位は「アカウント」ではなく「プロジェクト」
- 対応表は読み替えの入口。ただし思想が違うので1対1にはならない——それを体感するのがこの後のハンズオン

2. Dockerのおさらいとコンテナ起動

この章では Docker の要点をおさらいしたあと、実際に Web サーバーのコンテナイメージを作って・動かして・ブラウザで見るところまでやります。ここで作ったイメージを、次章以降で Cloud Run にデプロイしていきます。

Docker をしっかり学びたい人は [introduction-docker](#) にまともっています。今日は Cloud Run に必要な分だけ、駆け足で。

おさらい: 3つの登場人物

用語	一言でいうと	AWSでの馴染み
Dockerfile	イメージの設計図(テキストファイル)	CodeBuild でビルドしていたアレ
イメージ	アプリ+依存関係+OSライブラリを固めたスナップショット	ECR に push していたアレ
コンテナ	イメージを実行したプロセス	ECS タスクの中で動いていたアレ

ポイントは1つだけです: **イメージは不変(immutable)**。「環境ごと固めて配る」からこそ、ローカルでも Cloud Run でも ECS でも同じように動きます。この不変性が、後でやるロールバックやカナリアリリースの土台になります。

ハンズオン: Webサーバーのイメージを作る

アプリは TypeScript + [Hono](#) で書きます。Hono は Web 標準 API ベースの軽量な Web フレームワークで、Node.js でも Cloudflare Workers でも Deno でも同じコードが動きます。今回は Node.js 24 が TypeScript をそのまま実行できる(実行時に型が剥がされる)ため、**ビルドステップなし**で動かします。

TypeScript に詳しくなくても大丈夫です。書き換えるのは定数2行だけで、あとはコピー&ペーストで進められます。

1. 作業ディレクトリとファイルの作成

Cloud Shell エディタのターミナルで作業ディレクトリを作ります。

```
mkdir -p ~/cloudrun-handson/app/src
cd ~/cloudrun-handson/app
```

エディタで以下の4ファイルを作成してください(コピー&ペーストでOK)。このリポジトリの [code/app/](#) にも同じものがあります。

`package.json` — 依存関係は Hono 本体と Node.js アダプタの2つだけです。

```
{
  "name": "handson-app",
  "private": true,
  "type": "module",
  "engines": {
```

```

    "node": ">=24"
  },
  "scripts": {
    "start": "node src/index.ts"
  },
  "dependencies": {
    "@hono/node-server": "^1.19.0",
    "hono": "^4.9.0"
  }
}

```

`src/index.ts` — 小さなWebサーバーです。アクセスすると「メッセージ・リビジョン名・インスタンスID」を表示します。

```

import { serve } from "@hono/node-server";
import { Hono } from "hono";
import crypto from "node:crypto";

// =====
// ハンズオンで書き換えるのはこの2行です
// =====

const MESSAGE = "Hello, Cloud Run!";
const BG_COLOR = "#4285F4"; // v1: 青 / v2: "#EA4335" 赤 / v3: "#34A853" 緑

// コンテナ(インスタンス)が起動したタイミングで一意的なIDを生成する。
// オートスケールで何台に増えたかを見分けるために使う。
const INSTANCE_ID = crypto.randomUUID().slice(0, 8);

// stdout に1行JSONを吐くと Cloud Logging が構造化ログとして解釈する
const log = (severity: string, message: string, extra: Record<string, unknown> = {}) => {
  console.log(JSON.stringify({ severity, message, ...extra }));
};

const app = new Hono();

app.get("/", (c) => {
  log("INFO", "index accessed", { instance: INSTANCE_ID });
  // K_SERVICE / K_REVISION は Cloud Run が自動で注入する環境変数
  const service = process.env.K_SERVICE ?? "local";
  const revision = process.env.K_REVISION ?? "local";
  return c.html(`<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="utf-8">

```

```

<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Cloud Run Handson</title>
<style>
  body {
    margin: 0;
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    min-height: 100vh;
    background: ${BG_COLOR};
    color: #fff;
    font-family: sans-serif;
  }
  h1 { font-size: 3rem; margin: 0.5rem; }
  table { margin-top: 2rem; border-collapse: collapse; }
  td { padding: 0.3rem 1rem; border: 1px solid rgba(255,255,255,0.5); }
</style>
</head>
<body>
  <h1>${MESSAGE}</h1>
  <table>
    <tr><td>Service</td><td>${service}</td></tr>
    <tr><td>Revision</td><td>${revision}</td></tr>
    <tr><td>Instance</td><td>${INSTANCE_ID}</td></tr>
  </table>
</body>
</html>`);
});

```

// スクリプトから叩きやすいJSON版。カナリアの割合確認などに使う

```

app.get("/api", (c) =>
  c.json({
    message: MESSAGE,
    revision: process.env.K_REVISION ?? "local",
    instance: INSTANCE_ID,
  }),
);

```

// 1秒かかる重い処理のふり。負荷試験でオートスケールを観察するために使う

```

app.get("/heavy", async (c) => {
  await new Promise((resolve) => setTimeout(resolve, 1000));
  return c.json({ instance: INSTANCE_ID });
});

```

```
});

// Pub/Sub push サブスクリプションの受け口(発展編で使用)
app.post("/pubsub", async (c) => {
  const envelope = await c.req.json().catch(() => ({}));
  const data: string = envelope?.message?.data ?? "";
  const text = data ? Buffer.from(data, "base64").toString("utf-8") : "(empty)";
  log("INFO", `Pub/Sub message received: ${text}`);
  return c.body(null, 204);
});

// Cloud Run は環境変数 PORT でリッスンすべきポートを渡してくる
const port = Number(process.env.PORT ?? 8080);
serve({ fetch: app.fetch, port }, () => {
  log("INFO", `listening on port ${port}`);
});
```

Dockerfile

```
FROM node:24-slim

WORKDIR /app

COPY package.json ./
RUN npm install

COPY . ./

# Cloud Run は環境変数 PORT でリッスンすべきポートを渡してくる(デフォルト 8080)
ENV PORT=8080

# Node.js 24 は TypeScript をそのまま実行できる(型を剥がして実行される)
CMD ["node", "src/index.ts"]
```

`.dockerignore` — ローカルで `npm install` した場合に `node_modules` をイメージに混入させないためのファイルです。

```
node_modules
```

Dockerfile を上から読んでみましょう。ベースイメージ(`FROM`)に依存関係を重ね(`RUN npm install`)、コードを載せ(`COPY`)、起動コマンドを決める(`CMD`)。この積み重ねがそのままイメージのレイヤーになります。`package.json` だけ先

に COPY しているのは、コードを変更しても `npm install` のレイヤーキャッシュが効くようにするテクニックです(この後の章で効いてきます)。

1点だけ Cloud Run 固有の約束があります: 環境変数 `PORT` で渡されたポートを **listen** すること。ECS でいうタスク定義のポートマッピングにあたる取り決めが、この環境変数1つに集約されています。

2. イメージをビルドする

```
docker build -t handson-app:v1 .
```

手元に Node.js がなくても(`npm install` を一度も打っていないくても)ビルドできることに注目してください。依存解決も実行環境もすべてイメージの中で完結しています。`docker images` で `handson-app` ができていることを確認します。

3. コンテナを起動する

```
docker run --rm -p 8080:8080 handson-app:v1
```

4. ブラウザで見る

Cloud Shell の右上にある「ウェブでプレビュー」アイコン(目のマーク)→「ポート 8080 でプレビュー」をクリックします。

青い画面に「**Hello, Cloud Run!**」と表示されれば成功です。Revision と Service が `local` になっていることも見ておいてください——Cloud Run にデプロイすると、ここが自動で埋まります。

確認できたらターミナルで `Ctrl+C` を押してコンテナを止めます。

5. (余裕があれば)コンテナの中に入ってみる

```
docker run --rm -it handson-app:v1 bash
ls          # 自分の書いたコードと node_modules が /app に入っている
node -v     # イメージに焼き込まれた Node.js 24
exit
```

まとめ

- Dockerfile → イメージ → コンテナ、の流れを一周した
- イメージは不変。だから「どこでも同じに動く」し「前のバージョンに戻せる」
- Cloud Run との約束は「環境変数 `PORT` を listen する」だけ

今、あなたの手元(Cloud Shell)には動くイメージがあります。次章で Cloud Run の仕組みを知り、4章でこれをそのままクラウドに載せます。

3. Cloud Runとは

Cloud Run は「コンテナを渡すと、HTTPS の URL を返してくれる」フルマネージドなコンテナ実行基盤です。

- サーバー(VM・クラスタ)の管理は不要
- リクエストに応じて **0台から自動でスケール**し、使った分だけ課金
- デプロイの単位は普通のコンテナイメージ。ランタイムの縛りなし

AWSでいうと何?

App Runner・Fargate・Lambda のどれかに対応付けたいくなりますが、**どれも微妙に違います**。この「微妙に違う」がそのまま Cloud Run の魅力なので、表で整理します。

	Lambda	Fargate (ECS)	App Runner	Cloud Run
デプロイ単位	関数 / イメージ	イメージ	イメージ / ソース	イメージ / ソース
実行モデル	1リクエスト=1実行環境	常駐	常駐	1インスタンスで複数リクエストを並行処理 (デフォルト80)
スケールtoゼロ	○	×	×(最低1台)	○
課金	リクエスト処理時間	常時(起動時間)	常時+リクエスト	リクエスト処理中のみ (常時割当に変更も可)
実行時間上限	15分	なし	なし	60分(リクエストあたり)
HTTPS URL	要 API Gateway 等	要 ALB	○	○(自動発行)
WebSocket / gRPC / HTTP2	苦手(要別サービス)	○(要ALB/NLB設定)	×(WebSocket不可)	○
VPC	内で動く	内で動く	外(接続可)	外(接続可)

押さえてほしいポイントは3つです。

1. **Lambda の手軽さ × コンテナの自由度**。スケールtoゼロ・従量課金でありながら、中身はただのWebサーバー。Lambda 特有のハンドラ関数やランタイム制約がなく、手元で `docker run` できるものがそのまま動きます。
2. **1インスタンスが複数リクエストを同時に捌く**。Lambda は1リクエスト1環境なのでインスタンス数が爆発しがちですが、Cloud Run は「並行数(concurrency)」の分だけ1台で受けます。Webサーバーの常識がそのまま通用します。
3. **VPCが前提にない**。AWS では最初に VPC を作りますが、Cloud Run はデフォルトで VPC の外。DB接続などで必要になったときだけ VPC に「つなぎに行く」発想です。

Cloud Run の構成要素

ハンズオンで触る概念は3つだけです。

サービス (Service)

└─ リビジョン **rev-00003** ← トラフィック 100%
└─ リビジョン **rev-00002**
└─ リビジョン **rev-00001**

- **サービス**: URL を持つ論理単位。ECS でいう「サービス+ALB+ターゲットグループ」を1つにしたもの
- **リビジョン**: デプロイのたびに作られる**不変のスナップショット**(イメージ+環境変数+CPU/メモリ設定のセット)。過去のリビジョンは残り続ける
- **トラフィック分割**: どのリビジョンに何%流すかをサービスが制御。ロールバックもカナリアもこの仕組みの上に乗っている

「リビジョンが不変で、トラフィックを好きに振り分けられる」——2章でおさらいした**イメージの不変性がプラットフォームの機能にまで貫かれているのが Cloud Run の設計の美しさ**で、5章・6章でこれを体験します。

料金感

- リクエストを処理していない間は(デフォルト設定では)**CPU・メモリ課金なし**
- 無料枠: 月あたり vCPU 18万秒 / メモリ 36万GiB秒 / リクエスト200万件(2026年時点)
- 個人開発や社内ツールなら**実質無料で運用できる**ことが多い。「とりあえず公開しておく」が気軽にできる

制約・向かないもの

公平のために、向かないケースも知っておきましょう。

- 常時接続の重いステートフルワークロード(ゲームサーバー等) → GKE / GCE
- リクエスト起点でない常駐処理 → 常時割当CPU設定か、GKE
- 起動に数分かかる巨大アプリ → コールドスタートが辛い(7章で対策も扱います)

まとめ

- Cloud Run = コンテナを渡すと HTTPS URL が返ってくる。Lambda の運用感覚 × コンテナの自由度
- 概念は「サービス・リビジョン・トラフィック分割」の3つだけ
- 座学はここまで。**ここからは全部手を動かします**

4. Cloud Runへデプロイする

2章で作ったイメージを Artifact Registry(GAR)に push し、Cloud Run にデプロイして、世界中からアクセスできる HTTPS URL を手に入れます。

0. 環境変数の準備

この先の章でも使うので、まとめて設定しておきます(Cloud Shell が切断された場合はここから再実行してください)。

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION=asia-northeast1 # 東京リージョン
export REPO=handson
export IMAGE=${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO}/app
```

asia-northeast1 = 東京です。AWS の ap-northeast-1 と対応が覚えやすいですね。

1. Artifact Registry にリポジトリを作る

ECR と同じく、イメージの置き場をまず作ります。

```
gcloud artifacts repositories create ${REPO} \
  --repository-format=docker \
  --location=${REGION} \
  --description="Cloud Run handson"
```

docker コマンドが GAR に push できるよう、認証ヘルパーを設定します(ECR の `aws ecr get-login-password` に相当。こちらは一度設定すれば期限切れがありません)。

```
gcloud auth configure-docker ${REGION}-docker.pkg.dev
```

2. イメージにタグを付けて push する

GAR のイメージパスは リージョン-docker.pkg.dev/プロジェクト/リポジトリ/イメージ名:タグ という形式です。2章で作ったイメージにこのパスでタグを付け直して push します。

```
cd ~/cloudrun-handson/app
docker tag handson-app:v1 ${IMAGE}:v1
docker push ${IMAGE}:v1
```

push できたか確認してみましょう。


```
gcloud artifacts docker images list ${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO}
```

[コンソールの Artifact Registry](#) からも見えます。

3. Cloud Run にデプロイする

いよいよ本番です。このコマンド1つで、ECS なら ALB・ターゲットグループ・ACM 証明書・サービス・タスク定義を組み立てていた作業がすべて終わります。

```
gcloud run deploy handson-app \  
  --image ${IMAGE}:v1 \  
  --region ${REGION} \  
  --allow-unauthenticated
```

- `handson-app` がサービス名になります
- `--allow-unauthenticated` は「認証なしで公開」。デフォルトは IAM 認証必須(=閉じている)で、AWS と逆のデフォルトです

30秒ほどで完了し、Service URL: `https://handson-app-xxxxx.a.run.app` が表示されます。

4. アクセスして確認する

表示された URL をブラウザで開いてください。2章で Cloud Shell の中で見たのと同じ青い画面が、今度は本物のインターネット越しに表示されます。

2章との違いを見てみましょう:

- Service が `handson-app` に、Revision が `handson-app-00001-xxx` になっている(Cloud Run が注入する環境変数 `K_SERVICE` / `K_REVISION` をアプリが表示している)
- URL が最初から **HTTPS**。証明書の発行も更新も何もしていない

コンソールの [Cloud Run のページ](#) も開いてみてください。サービスの一覧、リビジョン、メトリクス、ログがすべて1画面に集約されています。

ふりかえり: 何をしなかったか

ここまでで「したこと」は リポジトリ作成 → push → deploy の3コマンドです。それより「しなかったこと」に注目してください。

- VPC・サブネット・セキュリティグループを作らなかった
- ロードバランサーもリスナーも作らなかった
- TLS 証明書を発行しなかった
- タスク定義(CPU/メモリ/ポート)を書かなかった ※デフォルト値で開始し、必要なら後から変える
- キャパシティ(台数)を決めなかった

1章で話した「ビルディングブロック vs SaaS的アプローチ」が、これで体感できたはずです。

5. 更新とロールバック

アプリを変更して2回目のデプロイを行い、「リビジョン」がどう積み重なるかを見ます。その後、**本番障害を想定して30秒でロールバック**します。

1. アプリを変更する (v2)

`src/index.ts` の上部にある2行を書き換えます。

```
const MESSAGE = "Hello, Cloud Run v2!";  
const BG_COLOR = "#EA4335"; // 赤
```

2. ビルドして **push** して **deploy**

2章・4章でやったことの繰り返しです。タグは `v2` にします。

```
cd ~/cloudrun-handson/app  
docker build -t ${IMAGE}:v2 .  
docker push ${IMAGE}:v2  
  
gcloud run deploy handson-app \  
  --image ${IMAGE}:v2 \  
  --region ${REGION}
```

2回目以降は `--allow-unauthenticated` などの設定を繰り返す必要はありません。サービスの設定は維持され、**変更したもの(イメージ)だけが新しいリビジョンとして記録**されます。

ブラウザをリロードしてください。画面が赤くなり、**Revision** が `handson-app-00002-xxx` に変わっていれば成功です。ダウンタイムなしで切り替わったことにも注目してください。

3. リビジョンの一覧を見る

```
gcloud run revisions list --service handson-app --region ${REGION}
```

00001 (青/v1)と 00002 (赤/v2)の両方が残っています。リビジョンは**イメージ+環境変数+リソース設定を固めた不変のスナップショット**で、デプロイのたびに積み重なっていきます。

AWSとの比較: ECS のタスク定義リビジョンに似ていますが、Cloud Run のリビジョンは「どこにトラフィックを流すか」の制御と一体化している点が違います。タスク定義+ALB加重ルーティング+デプロイ履歴が1つの概念になっているイメージです。

4. ロールバックする

ここで想定シナリオ: 「v2 をリリースしたらバグ報告が来た!すぐ戻りたい!」

AWS ならどうしますか? 前のタスク定義を指定してサービス更新、デプロイが走るのを待つ——数分かかります。Cloud Run では、**すでに存在する v1 のリビジョンにトラフィックを振り向けるだけです**。新しいデプロイは走りません。

まず v1 のリビジョン名を確認して:

```
gcloud run revisions list --service handson-app --region ${REGION}
```

トラフィックを 100% 戻します(`handson-app-00001-xxx` は自分のリビジョン名に置き換え):

```
gcloud run services update-traffic handson-app \  
  --region ${REGION} \  
  --to-revisions handson-app-00001-xxx=100
```

ブラウザをリロードしてください。**一瞬で青(v1)に戻ります**。

コンソールでも同じことができます: [Cloud Run](#) → サービス → 「リビジョン」タブ → 対象リビジョンのメニューから「このリビジョンにトラフィックを移行」。障害対応の現場では GUI でポチッと戻せるのは心強いです。

5. 次章に備えて v2 に戻しておく

ロールバック体験が終わったので、最新リビジョン(v2)に戻しておきます。

```
gcloud run services update-traffic handson-app \  
  --region ${REGION} \  
  --to-latest
```

ブラウザで赤(v2)に戻ったことを確認してください。

まとめ

- デプロイのたびに不変のリビジョンが積み重なる
- ロールバック = 過去リビジョンへのトラフィック切り替え。**再デプロイ不要、数秒で完了**
- 「イメージの不変性」がプラットフォーム機能として活きている

次章では、このトラフィック制御をもっと攻めた使い方——カナリアリリースに使います。

6. カナリアリリースとタグ付きURL

前章のトラフィック制御を一步進めて、**新バージョンを10%のユーザーにだけ出すカナリアリリース**と、トラフィックを流さずに**本番環境で動作確認できるタグ付きURL**を体験します。

AWSとの比較: これを ECS でやるには CodeDeploy のブルー/グリーン+ALB の加重ターゲットグループ、検証用リスナーの設定が必要でした。Cloud Run ではサービスに元から組み込まれた機能です。

1. v3 を作る

`src/index.ts` を再度書き換えます。

```
const MESSAGE = "Hello, Cloud Run v3!";  
const BG_COLOR = "#34A853"; // 緑
```

ビルドと push まで行います(まだ deploy はしません)。

```
cd ~/cloudrun-handson/app  
docker build -t ${IMAGE}:v3 .  
docker push ${IMAGE}:v3
```

2. トラフィックを流さずにデプロイする

`--no-traffic` を付けてデプロイすると、リビジョンは作られますがユーザーへのトラフィックは**1%も流れません**。さらに `--tag` を付けると、そのリビジョン専用のURLが発行されます。

```
gcloud run deploy handson-app \  
  --image ${IMAGE}:v3 \  
  --region ${REGION} \  
  --no-traffic \  
  --tag staging
```

出力に2つのURLが表示されます:

- 本番URL: `https://handson-app-xxxxx.a.run.app` → **まだ赤(v2)のまま**
- タグ付きURL: `https://staging---handson-app-xxxxx.a.run.app` → **緑(v3)**

両方をブラウザで開いて確認してください。**本番と同じ環境・同じ設定で、リリース前のバージョンだけを検証できるURL**が手に入りました。ステージング環境を別に組む代わりに、本番サービスの中に検証チャネルを持てるということです。

3. 10%だけ流す(カナリアリリース)

v3 の動作確認が取れた想定で、まず10%のユーザーにだけ出します。

```
gcloud run services update-traffic handson-app \  
  --region ${REGION} \  
  --to-tags staging=10
```

確認してみましょう。`/api` エンドポイントを20回叩いて、どのリビジョンが応答したかを集計します。

```
URL=$(gcloud run services describe handson-app --region ${REGION} --format 'value(status.url)')  
  
for i in $(seq 1 20); do curl -s ${URL}/api | jq -r .message; done | sort | uniq -c
```

おおよそ `v2` が18回・`v3` が2回、という比率になるはずです。ブラウザを何度もリロードして「たまに緑が出る」のを見るのも楽しいです。

実務ではこの状態でエラーレートやレイテンシのメトリクス(8章)を監視し、問題なければ段階的に割合を増やしていきます。

4. 100%に昇格する

```
gcloud run services update-traffic handson-app \  
  --region ${REGION} \  
  --to-latest
```

ブラウザで全リロードが緑(`v3`)になれば完了です。もし `v3` に問題が見つかったら?——前章でやった通り、`v2` に一瞬で戻せます。

まとめ

- `--no-traffic` + `--tag` で「デプロイ」と「リリース」を分離できる
- タグ付きURLで、本番環境そのものを使ったリリース前検証ができる
- カナリア → 段階的昇格 → 即ロールバック、が追加インフラなしで完結する

デプロイのたびに緊張するチームほど、この機能の価値は大きいはずです。

7. オートスケールを観察する

Cloud Run の非機能まわりの主役、オートスケールを実際に負荷をかけて観察します。あわせてコールドスタートとその対策も体験します。

オートスケールや非機能要件の背景を体系的に知りたい人は [クラウドを今から学ぶには](#) も参照してください。

Cloud Run のスケールの仕組み

Cloud Run は同時に処理しているリクエスト数を基準にインスタンス数を自動調整します。

必要インスタンス数 $\approx$ 同時リクエスト数 $\div$ concurrency (1台が同時に受ける数)

- concurrency はデフォルト80。**Lambda(1リクエスト=1環境)**と違い、**1台で複数リクエストを捌く**
- 0台までスケールインする(スケールtoゼロ)
- ECS のように CloudWatch アラーム+スケーリングポリシーを組む必要はなく、**設定不要で最初から動いている**

1. 観察しやすいように設定を変える

デフォルトの concurrency 80 だとなかなかスケールしないので、実験用に1台あたり10接続に絞り、上限を10台にします。

```
gcloud run services update handson-app \  
  --region ${REGION} \  
  --concurrency 10 \  
  --max-instances 10
```

この操作でも新しいリビジョンが作られます。リビジョン=イメージ+設定のスナップショットだからです。

2. 負荷をかける

負荷試験ツール [hey](#) を使います(Cloud Shell に入れるだけ)。

```
curl -sL -o ~/hey https://hey-release.s3.us-east-2.amazonaws.com/hey_linux_amd64  
chmod +x ~/hey
```

アプリの `/heavy` は1秒かかる擬似的に重いエンドポイントです。同時50接続で30秒間叩きます。

```
URL=$(gcloud run services describe handson-app --region ${REGION} --format 'value(status.url)')  
  
~/hey -z 30s -c 50 ${URL}/heavy
```

同時50接続 ÷ concurrency 10 = **5台前後までスケールアウトする**はずです。

3. 観察する

負荷をかけている間(および直後)に、2つの方法で観察します。

コンソールで: [Cloud Run](#) → handson-app → 「指標」タブ。「コンテナ インスタンスの数」がゼロから跳ね上がり、数分後にまたゼロに戻っていく様子が見えます。リクエスト数・レイテンシ(p50/p95/p99)・CPU使用率も何も設定していないのに最初から揃っています。

手元で: `/api` はインスタンスごとに違う `instance ID` を返します。何台に分散しているか数えてみましょう。

```
for i in $(seq 1 30); do curl -s ${URL}/api | jq -r .instance; done | sort | uniq -c
```

複数の instance ID が出れば、リクエストが複数台に分散している証拠です。負荷が落ち着いた数分後に同じコマンドを打つと、1台(やがて0台)に戻ります。

4. コールドスタートと min-instances

スケールtoゼロの代償がコールドスタートです。0台の状態への最初のリクエストは、コンテナ起動を待つ分だけ遅くなります。

10分ほど放置したあと(または講師の説明を聞いたあと)、1発目のレスポンスタイムを計ってみてください。

```
curl -s -o /dev/null -w "time_total: %{time_total}s\n" ${URL}/  
curl -s -o /dev/null -w "time_total: %{time_total}s\n" ${URL}/
```

1回目だけ遅い(このアプリは軽いので数百ms程度。重いアプリでは数秒～)のがわかります。対策は常時1台温めておくことです。

```
gcloud run services update handson-app \  
  --region ${REGION} \  
  --min-instances 1
```

トレードオフ: min-instances を設定した分はアイドル時も課金されます。「完全従量制(コールドスタートあり)」と「最低台数の固定費(コールドスタートなし)」を、サービスごとにダイアルで選べるのが Cloud Run の良いところです。

Lambda の Provisioned Concurrency と ECS の常駐、両方の選択肢が1つのサービスに入っていると考えてください。

確認できたら、課金を避けるため戻しておきます。

```
gcloud run services update handson-app \  
  --region ${REGION} \  
  --min-instances 0
```


まとめ

- スケールの基準は「同時リクエスト数 ÷ concurrency」。設定ゼロで動く
- スケールtoゼロ ⇔ コールドスタートはトレードオフ。`min-instances` で調整
- `max-instances` は暴走課金・下流(DB)保護の安全弁としても重要

8. ログとメトリクスをのぞく

短い章です。ここまでのハンズオンで一度も「ログの設定」をしていないのに、実はすべて記録されています。それを見に行きます。

1. ログを見る

[Cloud Run のコンソール](#) → `handson-app` → 「ログ」タブを開いてください。

- コンテナが `stdout/stderr` に出しただけのログが、すべて収集されている
- アプリが出していた `{"severity": "INFO", "message": "index accessed", ...}` という1行JSONが構造化ログとして解釈され、severity で色分けされている

CLI からも見られます:

```
gcloud run services logs read handson-app --region ${REGION} --limit 20
```

AWSとの比較: CloudWatch Logs へ送るには `awslogs` ログドライバやロググループの設定、IAM 権限が必要でした。Cloud Run ではコンテナが `stdout` に書く、以上です。構造化ログにしたければ1行JSONにするだけで、フィールド検索・severityフィルタが効くようになります。

2. Logs Explorer で検索する

「ログ」タブの上部から「ログ エクスプローラで表示」を開くと、プロジェクト全体のログを横断検索できます。クエリ欄に以下を入れてみてください:

```
resource.type="cloud_run_revision"
resource.labels.service_name="handson-app"
jsonPayload.message="index accessed"
```

アプリが出した構造化ログだけに絞ります。 `jsonPayload.instance` など、 `log()` 関数で自分が入れたフィールドもそのまま検索条件に使えます。

3. メトリクスを見る

「指標」タブ(7章でも見ました)には以下が最初から揃っています:

- リクエスト数 / レイテンシ(p50, p95, p99)
- コンテナインスタンス数 / 課金対象インスタンス時間
- CPU / メモリ使用率

実務では、6章のカナリアリリース中にこの画面を開き、「新リビジョンだけエラー率が上がっていないか」を見ながら昇格判断をします。リビジョン別にメトリクスをフィルタできることも確認してみてください。

まとめ

- ログもメトリクスも「設定する」のではなく「最初からある」
- アプリ側の作法は「stdout に(できれば1行JSONで)書く」だけ
- 1章で話した SaaS 的アプローチは、オブザーバビリティにも一貫している

9. 締め: Dockerfileすら書かないデプロイ

最後に、今日学んだ手順を巻き戻すような機能を紹介します。ソースコードだけで **Cloud Run** にデプロイする方法です。

ソースデプロイ

`--image` の代わりに `--source` を指定すると、Cloud Build がソースからイメージをビルドして GAR に push し、そのままデプロイまで行います。

```
cd ~/cloudrun-handson/app
gcloud run deploy handson-app-src \
  --source . \
  --region ${REGION} \
  --allow-unauthenticated
```

数分待つと、別サービス `handson-app-src` として同じアプリが公開されます。

このディレクトリには Dockerfile があるのでそれが使われますが、**Dockerfile** を消しても動きます。その場合は [Cloud Native Buildpacks](#) が `package.json` を見て「Node.js アプリだな」と判断し、`npm start` で起動する良い感じのイメージを自動生成します。

AWSとの比較: App Runner のソースデプロイに相当しますが、できあがるものは今日ずっと触ってきた「普通の Cloud Run サービス」です。カナリアも、タグ付きURLも、スケール設定も全部同じように使えます。

では、なぜ今日 Dockerfile から始めたのか

「最初からこれを教えてくれれば良かったのに」と思うかもしれません。でも順番には意図があります。

- ソースデプロイは中でやっていることが今日の手順そのものです(build → push → deploy)。仕組みを知った上で使う自動化と、知らずに使う魔法は違います
- 実務では Dockerfile を書く場面が必ず来ます(依存OSパッケージ、マルチステージビルド、社内ベースイメージ...)
- 逆に、プロトタイプや社内ツールなら「Dockerfile なしでとりあえず公開」で十分なことも多い

手軽さの階段を自分で選べるのが Cloud Run です:

```
gcloud run deploy --source .           # ソースだけ(Buildpacks におまかせ)
gcloud run deploy --source .           # ソース+ Dockerfile(ビルドはおまかせ)
docker build && push && deploy          # 全部自分で制御(今日やったこと)
+ Cloud Build トリガー                 # git push で自動デプロイ(CI/CD)
```

今日のまとめ

1. **AWS**はビルディングブロック、**Google Cloud**は**SaaS**的アプローチ — deploy 1コマンドの裏で、LB・証明書・スケール・ログ収集を「しなかった」

2. イメージの不変性がプラットフォームまで貫かれている — リビジョン、ロールバック、カナリア、タグ付きURL

3. 従量課金と常駐のダイヤルを回せる — スケールtoゼロ、concurrency、min/max-instances

AWS に帰っても、この目線は使えます。「この構成、Cloud Run 的な発想ならどう簡単にできるか?」「App Runner や Lambda で十分では?」——別のクラウドを知ることは、いま使っているクラウドをより良く使うことにつながります。

時間に余裕があれば[発展編](#)へ。終わったら必ず[後片付け](#)をしてください。

10. 発展編

3時間版で扱う(または2時間版では講師デモとして見せる)コンテンツです。Cloud Run 単体の体験から一歩進んで、他のサービスとの連携のしやすさと、「Webサーバー」以外のワークロードを体験します。

節	内容	形式	所要時間
10-1. Pub/Subとつなぐ	イベント駆動アーキテクチャを3コマンドで	ハンズオン	20分
10-2. WebSocketチャット	ALBなしでWebSocketが動く	デモ(全員参加)	10分
10-3. Cloud Run Jobs	リクエスト駆動ではないバッチ処理	デモ or ハンズオン	10分

ここで伝えたいこと

4～9章で見てきた Cloud Run は「Webサービスの実行基盤」でした。発展編で見るのはその周辺です。

- **Pub/Sub 連携** — Lambda + SQS で組んでいたイベント駆動が、「普通のHTTPサーバーにPOSTが飛んでくる」だけの世界になる
- **WebSocket** — 「サーバーレスでは無理」と思われがちなワークロードが普通に動く
- **Jobs** — 同じイメージ・同じ開発体験のまま、バッチにも使える

いずれも「Cloud Run が特別多機能」というより、「ただのコンテナ・ただのHTTP」という抽象を守っているから、何とでもつながるという話です。

10-1. Pub/Subとつなぐ

Cloud Run を Pub/Sub のサブスクライバーにして、イベント駆動アーキテクチャを体験します。

AWSでの構成と比べる

「S3にファイルが置かれたら処理する」「注文イベントを非同期で処理する」——AWS なら SNS + SQS + Lambda(またはSQSポーリングのECS)で組む定番パターンです。Lambda には専用のハンドリングネチャがあり、SQS はポーリング設定やバッチサイズの調整が必要でした。

Google Cloud では **Pub/Sub** がサブスクライバーのHTTPエンドポイントに直接 **POST** してくれます(push型)。受け側は「POSTを受けるだけの普通のWebサーバー」でよく、つまり今日デプロイしたアプリがそのまま使えます。

実は2章で作った `src/index.ts` には、こっそり受け口を仕込んであります:

```
// Pub/Sub push サブスクリプションの受け口(発展編で使用)
app.post("/pubsub", async (c) => {
  const envelope = await c.req.json().catch(() => ({}));
  const data: string = envelope?.message?.data ?? "";
  const text = data ? Buffer.from(data, "base64").toString("utf-8") : "(empty)";
  log("INFO", `Pub/Sub message received: ${text}`);
  return c.body(null, 204);
});
```

1. トピックとpushサブスクリプションを作る

```
gcloud pubsub topics create handson-topic

URL=$(gcloud run services describe handson-app --region ${REGION} --format 'value(status.url)')

gcloud pubsub subscriptions create handson-sub \
  --topic handson-topic \
  --push-endpoint ${URL}/pubsub
```

これで配線は完了です。SQSキューの作成、イベントソースマッピング、IAMロール、バッチ設定.....に相当する作業はありません。

2. メッセージを発行してみる

```
gcloud pubsub topics publish handson-topic --message "Hello from Pub/Sub"
```

数秒待ってからログを確認します:

```
gcloud run services logs read handson-app --region ${REGION} --limit 10
```

Pub/Sub message received: Hello from Pub/Sub が出ていれば成功です。8章の Logs Explorer で見ると、構造化ログとして届いているのも確認できます。

3. ここで想像してほしいこと

- このエンドポイントが重い処理(画像変換、集計、通知送信)だったら? → **Pub/Sub** が流量を受け止め、**Cloud Run** が処理量に応じてスケールアウトする。バックプレッシャー付きの非同期処理基盤が、この3コマンドでできています
- 処理が失敗したら(2xx以外を返したら)? → Pub/Sub が自動でリトライします。デッドレタートピックも設定できます
- スケールtoゼロと組み合わせると? → イベントが来たときだけ起動して課金される、Lambda 的な使い方が「普通のWebサーバー」のままできます

本番に向けた補足

ハンズオンでは簡単のため認証なし(`--allow-unauthenticated` なサービス)に push しましたが、本番では以下のようになります:

- サービスを `--no-allow-unauthenticated` にして、Pub/Sub 用のサービスアカウントに `roles/run.invoker` を付与
- サブスクリプションに `--push-auth-service-account` を指定すると、Pub/Sub が OIDC トークン付きで POST してくれる

「サービス間の認証を IAM とIDトークンでやる」のは Cloud Run 全般のパターンで、内部マイクロサービス間の呼び出しも同じ仕組みで守れます(ALB の内部リスナーやセキュリティグループの代わりに、IAM で「誰が呼べるか」を制御するイメージです)。

さらに Pub/Sub 以外にも、**Eventarc** を使うと Cloud Storage のファイル作成や監査ログなど60以上のイベントソースから Cloud Run を起動できます(EventBridge 相当)。**Cloud Scheduler**(cron)や **Cloud Tasks**(遅延・レート制御付きキュー)も、同じく「HTTPエンドポイントを叩く」という形で Cloud Run と連携します。

後片付け(この節の分)

```
gcloud pubsub subscriptions delete handson-sub
gcloud pubsub topics delete handson-topic
```


10-2. WebSocketチャット

「サーバーレスでWebSocketは無理」という常識(Lambda 単体では扱えず、API Gateway の WebSocket API + DynamoDB で接続管理...)を、Cloud Run があっさり覆すのを見ます。

進め方: イベントでは講師がデプロイ済みのURLを画面に映し、**参加者全員がスマホやPCから同じチャットに参加**します。自分でもデプロイしたい人向けの手順も載せています(2章のスキルで全部できます)。

なぜ Cloud Run で WebSocket が動くのか

Cloud Run のインスタンスは「普通のHTTPサーバーのプロセス」なので、コネクションを張りっぱなしにできます。特別な設定はほぼ不要で、注意点は3つだけです:

- リクエストタイムアウト(デフォルト5分、最大60分)がコネクションにも適用される → `--timeout` を伸ばす。切断時はクライアントが再接続する設計にする
- 接続はインスタンスのメモリに紐づく → 複数インスタンス間で状態を共有するには Memorystore (Redis) 等が必要
- 再接続時に同じインスタンスへ戻したい → `--session-affinity` を付ける

デプロイ手順(講師 or 自分でやりたい人向け)

コードはこのリポジトリの [code/websocket/](#) にあります。本編と同じ TypeScript + Hono 製(`@hono/node-ws` を追加)の小さなチャットサーバーで、受け取ったメッセージを接続中の全クライアントにブロードキャストするだけのものです。

```
cd ~/cloudrun-handson
git clone https://github.com/y-ohgi/handson-CloudRun.git
cd handson-CloudRun/code/websocket

docker build -t ${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO}/chat:v1 .
docker push ${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO}/chat:v1

gcloud run deploy handson-chat \
  --image ${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO}/chat:v1 \
  --region ${REGION} \
  --allow-unauthenticated \
  --timeout 3600 \
  --session-affinity \
  --max-instances 1
```

`--max-instances 1` は、接続管理をインスタンスのメモリで済ませているこのデモアプリの都合です。1台に固定することで「全員が同じメモリ空間につながる」状態を作っています。

体験する

発行されたURLを全員で開き、メッセージを送り合ってみてください。リアルタイムに全員の画面へ配信されます。

いま起きていることを整理すると:

- ロードバランサーも API Gateway も立てていない
- 接続管理のための DynamoDB 相当も書いていない
- コードはローカルで `docker run` してもそのまま動く、教科書通りの WebSocket サーバー

実務で使うなら

- 状態共有: インスタンス間のブロードキャストは **Memorystore (Redis) の Pub/Sub** を挟むのが定番。そうすれば `--max-instances` の制限は外せます
- gRPC(双方向ストリーミング含む)や Server-Sent Events も同様にサポートされています。「**HTTPでできることは大体できる**」と覚えておけばOKです

10-3. Cloud Run Jobs

ここまで扱ってきたのは Cloud Run の「サービス」——HTTPリクエストを受けるワークロードでした。もう1つの顔が **Jobs**: リクエストを受けず、**処理が終わったら終了する**ワークロードです。バッチ処理、DBマイグレーション、定期集計などに使います。

AWSでいうと: ECS の RunTask、AWS Batch、(15分制限がなければ)Lambda あたりの守備範囲です。

サービスとの違い

	サービス	Jobs
起動のきっかけ	HTTPリクエスト	実行命令(手動 / スケジュール / API)
終わり方	常に待ち受け	プロセスが exit したら完了
PORT の listen	必須	不要
並列実行	オートスケール	タスク数・並列数を指定(<code>--tasks</code>)
リトライ	(Pub/Sub等の呼び出し側で)	<code>--max-retries</code> で組み込み

重要なのは、**同じイメージ・同じレジストリ・同じ開発体験のまま**、Web とバッチの両方をカバーできることです。

1. ジョブを作って実行する

今日作ったイメージを流用し、起動コマンドだけ上書きしてジョブにします。

```
gcloud run jobs create handson-job \  
  --image ${IMAGE}:v3 \  
  --region ${REGION} \  
  --command node \  
  --args "-e,console.log('Hello from Cloud Run Jobs')"
```

実行します。 `--wait` で完了まで待ちます。

```
gcloud run jobs execute handson-job --region ${REGION} --wait
```

ログを見てみましょう:

```
gcloud beta run jobs logs read handson-job --region ${REGION}
```

コンソールの [Cloud Run](#) では「ジョブ」タブに実行履歴・成功/失敗・ログが並びます。

2. 定期実行(cron)にする

Cloud Scheduler(EventBridge Scheduler 相当)から叩けば cron バッチになります。コンソールのジョブ詳細画面から「トリガー」→「スケジューラー トリガーを追加」で GUI から設定できます。

```
# 例: 毎朝9時(JST)に実行するトリガー
gcloud scheduler jobs create http handson-job-schedule \
  --location ${REGION} \
  --schedule "0 9 * * *" \
  --time-zone "Asia/Tokyo" \
  --uri "https://run.googleapis.com/v2/projects/${PROJECT_ID}/locations/${REGION}/jobs/handson-
job:run" \
  --oauth-service-account-email $(gcloud iam service-accounts list --format 'value(email)' --filter
'displayName:"Compute Engine default service account"')
```

こちらはコマンドが少し長いので、イベントでは GUI での設定を見せるだけで十分です。

3. 並列実行

Jobs の面白いところは組み込みの並列実行です。例えば「10万件のデータを100タスクで分担して処理」のような使い方ができます:

```
gcloud run jobs update handson-job --region ${REGION} --tasks 5
gcloud run jobs execute handson-job --region ${REGION} --wait
```

各タスクには `CLOUD_RUN_TASK_INDEX` / `CLOUD_RUN_TASK_COUNT` という環境変数が渡されるので、「自分は何番目か」で担当範囲を割り出せます。AWS Batch の配列ジョブ相当が、追加サービスなしで使えます。

後片付け(この節の分)

```
gcloud scheduler jobs delete handson-job-schedule --location ${REGION} --quiet # 作った場合のみ
gcloud run jobs delete handson-job --region ${REGION} --quiet
```

99. 後片付け

お疲れさまでした!課金が発生し続けるリソースを片付けます。

おすすめ: プロジェクトごと削除

ハンズオン専用プロジェクトで進めた場合は、これが一番確実です。

```
gcloud projects delete ${PROJECT_ID}
```

(30日間は復元可能な状態で保留されたあと、完全に削除されます)

個別に削除する場合

既存プロジェクトを使った場合はこちら。

```
# Cloud Run サービス
gcloud run services delete handson-app --region ${REGION} --quiet
gcloud run services delete handson-app-src --region ${REGION} --quiet # 9章を実施した場合
gcloud run services delete handson-chat --region ${REGION} --quiet # 10-2を実施した場合

# Cloud Run ジョブ(10-3を実施した場合)
gcloud run jobs delete handson-job --region ${REGION} --quiet

# Pub/Sub(10-1を実施した場合)
gcloud pubsub subscriptions delete handson-sub
gcloud pubsub topics delete handson-topic

# Artifact Registry(イメージの保管料がかかるため忘れずに)
gcloud artifacts repositories delete ${REPO} --location ${REGION} --quiet
```

課金ポイントの整理: Cloud Run はスケールtoゼロなので、`min-instances` を0に戻してあれば放置してもほぼ課金されません。継続課金になり得るのは **Artifact Registry** のストレージ(無料枠0.5GB超過分)と **min-instances** です。

消し忘れが心配な人へ

[課金レポート](#)で数日後に確認するか、予算アラート(Budgets & alerts)を設定しておくで安心です。AWS の Budgets と同じ感覚で使えます。

続けて学びたい人へ

- [Cloud Run 公式ドキュメント](#) — 品質が高く日本語も充実
- [introduction-docker](#) — Docker を基礎から
- [クラウドを今から学ぶには](#) — 非機能要件の考え方
- 次の一歩: Cloud Build トリガーで「git push したら自動デプロイ」、Secret Manager 連携、`--no-allow-unauthenticated` + IAM でのサービス間認証、Cloud SQL 接続あたりが実務への近道です